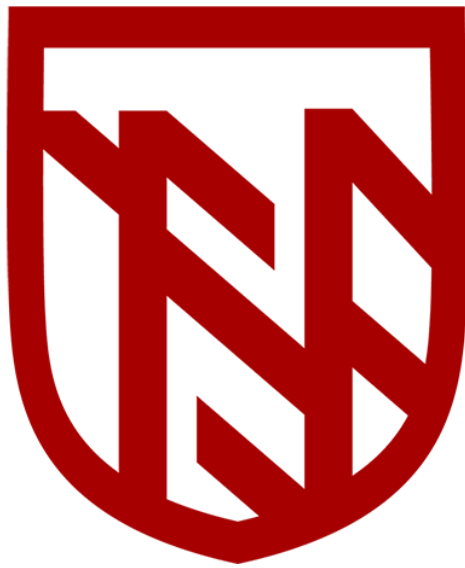




EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



**TEC
SAN
PEDRO®**

PROYECTO ANÁLISIS INDIVIDUAL

TEC SAN PEDRO DE LAS COLONIAS

Equipo:

Sergio Garza Favela

Docente:

Ruth Aivi Chavez Rodriguez

Área:

Fundamentos de ing en software



Análisis Individual de Ingeniería de Software: Caso C01

1. Presentación del caso y consecuencias para usuarios, equipo y organización

El caso C01 (Sistema de citas de una clínica) expone la situación de una clínica pequeña que implementa un sistema web para la gestión de citas médicas. Aunque la interfaz permite seleccionar los campos clave (paciente, médico, fecha y horario) y aparenta funcionar correctamente al confirmar operaciones sin errores visibles, el proceso carece de controles técnicos y organizacionales robustos.

Las consecuencias de estas deficiencias impactan de manera directa a tres niveles:

- Usuarios (Pacientes y recepcionistas): Sufren por la duplicidad de citas, malentendidos en los horarios y la pérdida de confianza en la atención médica, mientras que el personal administrativo gasta tiempo resolviendo confusiones presenciales o telefónicas.
- Equipo de desarrollo: Trabaja bajo presión constante y de manera reactiva, implementando parches urgentes directamente en producción sin un control de calidad adecuado ni respaldos confiables.
- Organización: Experimenta fricciones y discusiones constantes entre áreas, ineficiencia operativa, pérdida de tiempo laboral en tareas correctivas y un riesgo latente de vulneración en la integridad de los datos de salud.

2. Cinco problemas identificados

Problema 1: Falta de control de concurrencia en la reserva de turnos

- Causa: Ausencia de mecanismos de bloqueo a nivel de base de datos o transacciones atómicas que impidan que dos transacciones simultáneas accedan al mismo intervalo de tiempo.
- Consecuencia: Duplicidad de citas en el mismo horario, lo que genera colisiones en la atención médica y conflictos directos entre pacientes y recepcionistas.
- Práctica de Ingeniería de Software: Gestión de concurrencia y control de transacciones (ACIID) a nivel de arquitectura de datos.
- Evidencia: *"Dos recepcionistas pueden reservar el mismo horario casi al mismo tiempo"* y *"aumentan las citas duplicadas"*.



Problema 2: Ausencia de trazabilidad y registro de auditoría (Log de cambios)

- Causa: El software no registra las metainformaciones de las operaciones (quién realizó la inserción, modificación o cancelación, ni la estampa de tiempo exacta).
- Consecuencia: Imposibilidad de deslindar responsabilidades o rastrear el origen de un error cuando un cambio se gestiona por canales informales.
- Práctica de Ingeniería de Software: Registro de auditoría y control de trazabilidad en sistemas transaccionales.
- Evidencia: *"Tampoco existe un registro claro de quién modificó o canceló una cita".*

Problema 3: Despliegues directos a producción sin un ciclo de pruebas formal

- Causa: Ausencia de ambientes separados (desarrollo, pruebas, producción) y falta de un pipeline de integración y despliegue continuo.
- Consecuencia: Alta propensión a introducir errores críticos no detectados, colapsando el servicio ante urgencias operativas.
- Práctica de Ingeniería de Software: Pruebas de software estructuradas (unitarias, de integración y de sistema) y gestión de despliegues.
- Evidencia: *"El equipo desarrollador publica cambios directamente en producción cuando la clínica reporta un problema urgente. Las pruebas se limitan a comprobar que la pantalla abra y que sea posible guardar una cita".*

Problema 4: Gestión informal de cambios y dependencia de la comunicación verbal

- Causa: Falta de un protocolo formal de control de cambios y de un canal unificado para la gestión de solicitudes y modificaciones operativas.
- Consecuencia: Desalineación entre lo que refleja el sistema y la realidad operativa de la clínica, aumentando las discusiones internas.
- Práctica de Ingeniería de Software: Gestión de configuración y control de requerimientos.
- Evidencia: *"Cuando una cita cambia, el personal suele avisar por mensajería y no siempre actualiza el sistema", generando "discusiones entre áreas".*

Problema 5: Inexistencia de políticas de respaldo y recuperación ante desastres

- Causa: Falta de planificación técnica para la salvaguarda de la información y de simulacros periódicos de restauración.
- Consecuencia: Vulnerabilidad ante fallas de hardware, errores humanos o corrupción de datos, con la consecuente pérdida definitiva de información crítica.
- Práctica de Ingeniería de Software: Planificación de respaldo, recuperación (Backup & Disaster Recovery) y gestión de riesgos.
- Evidencia: *"No hay un procedimiento acordado para respaldar la información, recuperar una versión anterior ni capacitar al personal".*



3. Tres acciones prioritarias

1. **Orden 1: Implementar control de concurrencia y transacciones atómicas en la base de datos.**
 - *Justificación:* Es la falla más crítica que afecta directamente la operación diaria y la integridad de las citas médicas.
 - *Costo o limitación:* Requiere modificar la lógica de negocio y las consultas SQL para aplicar bloqueos optimistas o pesimistas, con un esfuerzo técnico moderado.
 - *Forma de comprobar la mejora:* Realizar pruebas de estrés simulando reservas concurrentes exactas para verificar que el sistema rechace o ponga en espera la segunda solicitud sin duplicar registros.
2. **Orden 2: Establecer un entorno de pruebas aislado y un protocolo formal de despliegues.**
 - *Justificación:* Detiene la mala práctica de programar y corregir directamente en producción, reduciendo drásticamente la introducción de bugs.
 - *Costo o limitación:* Inversión de tiempo en configurar servidores de prueba y capacitar al equipo en el uso de control de versiones.
 - *Forma de comprobar la mejora:* Auditar que ningún cambio llegue a producción sin pasar por una lista de verificación de pruebas funcionales previas.
 -
3. **Orden 3: Diseñar e implementar un sistema de respaldo automatizado y bitácoras de auditoría.**
 - *Justificación:* Garantiza la recuperación de datos ante imprevistos y otorga trazabilidad sobre el uso del sistema.
 - *Costo o limitación:* Consumo de espacio de almacenamiento adicional y definición de permisos de usuario.
 - *Forma de comprobar la mejora:* Ejecutar una prueba mensual de restauración de base de datos en un ambiente limpio y revisar los reportes de auditoría generados por el sistema.

4. Reflexión personal

Este análisis me ha permitido comprender que la calidad de un sistema informático va mucho más allá de una interfaz funcional o de cumplir con los requerimientos superficiales de una pantalla. Con frecuencia, la presión operativa o la urgencia por entregar resultados pueden inclinar al equipo hacia prácticas improvisadas, como la aplicación de correcciones directas en producción. Sin embargo, estas acciones solo postergan y agravan los problemas estructurales. Como estudiante de ingeniería, reconozco la importancia de adoptar un enfoque metódico, priorizando siempre la arquitectura robusta, la integridad de la información y la planeación rigurosa. A partir de esta experiencia, me propongo transformar



mi manera de desarrollar software, sustituyendo las soluciones reactivas por procesos disciplinados que garanticen la estabilidad, la seguridad y la confiabilidad de los sistemas a largo plazo.

5. Fuentes consultadas y declaración de uso de IA

- **Fuentes consultadas:**
 - Documento base de la actividad: *Fundamentos de Ingeniería de Software*
- [https://es.wikipedia.org/wiki/Transacci%C3%B3n_\(inform%C3%A1tica\)](https://es.wikipedia.org/wiki/Transacci%C3%B3n_(inform%C3%A1tica))
- https://es.wikipedia.org/wiki/Computaci%C3%B3n_concurrente
- https://es.wikipedia.org/wiki/Multiversion_concurrency_control
- https://es.wikipedia.org/wiki/Esquema_de_una_base_de_datos
- https://es.wikipedia.org/wiki/Modelo_de_base_de_datos
- https://es.wikipedia.org/wiki/Prueba_de_software
- https://es.wikipedia.org/wiki/Integraci%C3%B3n_continua
- https://es.wikipedia.org/wiki/Control_de_versiones
- https://es.wikipedia.org/wiki/Gesti%C3%B3n_de_configuraci%C3%B3n